



Final-Year Project — AML Investigation Agent (GenAI + Agentic RAG)



What: a GenAI **agent that investigates AML alerts** — given a flagged account/transaction it gathers the transaction history + graph, scores risk (**your FundFlow engine, wired in as a tool**), retrieves the relevant real AML regulations/typologies (**RAG with citations**), and drafts a **Suspicious Activity Report (SAR)** with cited justification + a confidence flag.

Why it upgrades your resume: fills your deployed-GenAI gap, serves **Resume A** (finance/AML → banks & GCCs) *and* **Resume C** (MLE/GenAI), and **levels up FundFlow instead of repeating it**.



The dataset question — settled (this was your worry)

Your instinct is half-right, so let's be precise:

- **True:** raw, real *customer* bank-transaction data is confidential and effectively unavailable — privacy law forbids it. **Nobody gets it.** That's exactly why the *entire* AML industry and all academic research run on **synthetic data**. So synthetic is the norm, **not a weakness** — you just disclose it honestly (as you did with FundFlow).
- **Wrong:** "there are no datasets." There are purpose-built AML datasets far better than PaySim, and even a **real-world** graph option:

Dataset	Real / Synth	Why use it
IBM Transactions for AML (AMLSim)	Synthetic	Primary pick. Bank accounts + transactions + alerts, with labeled laundering typologies. Same account/transfer world as FundFlow → your engine transfers
SAML-D	Synthetic	12 features, 28 laundering typologies , high-risk countries/payment types, "Is Suspicious" label — adds typology richness
Elliptic++	Real	203k real Bitcoin transactions + 822k wallets, labeled illicit/licit — a genuine real-world graph if you want to silence the "synthetic" worry entirely
SynthAML	Synth (from real)	Built on a real Danish bank's data: 20k alerts + 16M transactions

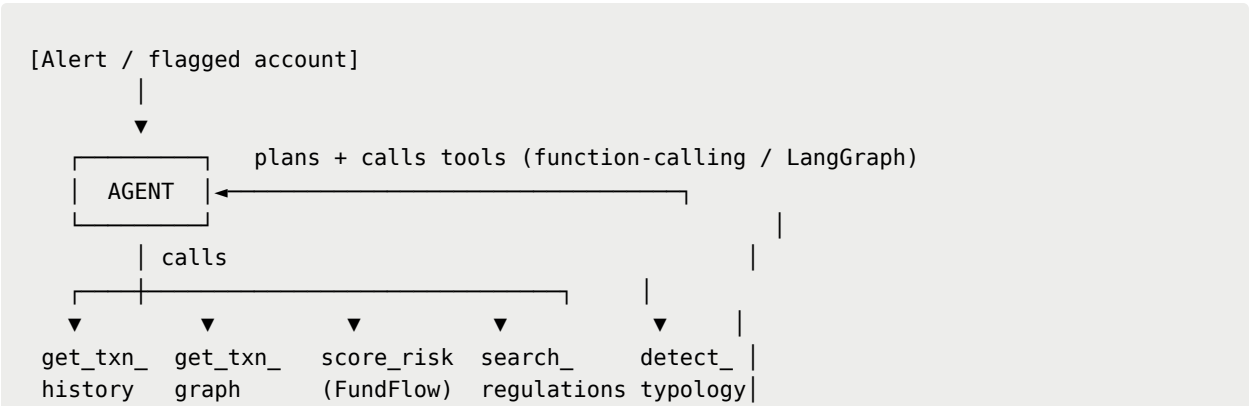


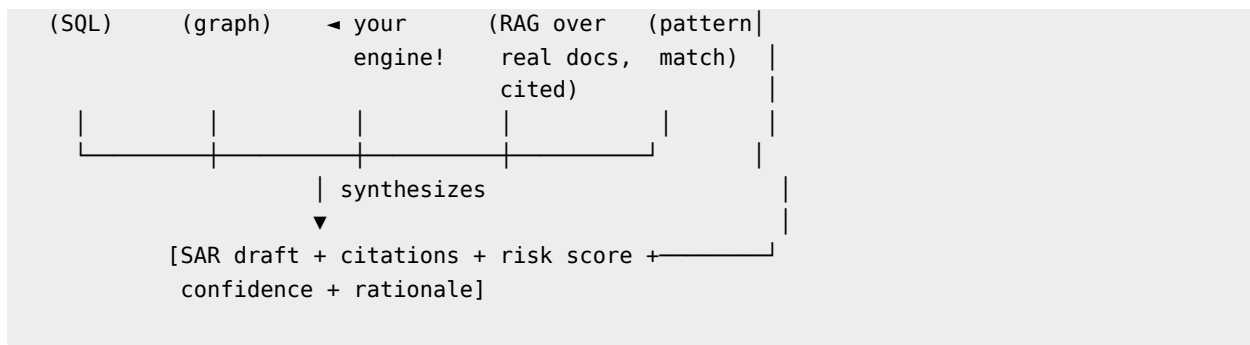
The real-world part is your RAG corpus, and it's genuinely real: [FATF Recommendations](#), [FinCEN threat-pattern / trend reports](#), and the [RBI KYC/AML Master Direction](#). Your agent retrieves over **real regulations** — that's the impressive, defensible layer. The transaction data is just fuel; the value of this project is the agent + RAG + eval + deployment.

Recommendation: transactions = **IBM AMLSim** (+ SAML-D for typology variety); RAG = **real FATF/FinCEN/RBI docs**. **Don't reuse PaySim** — a different dataset keeps this from feeling like "FundFlow 2."



Architecture





Components

- **Data layer:** AMLSim transactions/accounts in SQLite/Postgres; account graph in NetworkX (or a lite graph DB).
- **Agent tools:** `get_transaction_history`, `get_transaction_graph`, `score_risk` (FundFlow XGBoost/graph engine retrained on AMLSim, exposed as a callable tool), `search_regulations` (RAG, returns cited passages), `detect_typology` (structuring / layering / smurfing match).
- **Orchestration:** LLM with function-calling / LangGraph — plan → gather → reason → draft.
- **RAG:** chunk + embed the real AML docs, vector store (Chroma/FAISS), retrieve-with-citations.
- **Guardrails:** enforce citations, refuse on insufficient evidence, hallucination check.
- **Backend:** FastAPI you write and can defend, Dockerized. **Frontend:** Streamlit/Gradio investigation console. **Deploy:** Render/Railway/HF Spaces.
- **Eval:** RAGAS (faithfulness, answer/context relevance, context recall) + retrieval hit-rate + a small labeled alert set for investigation quality.

 **Scoped build plan (MVP first — this is a full-semester FYP)**



Build in this order. **Phases 0–2 = a working, deployed MVP** — ship that before touching the fancy parts. Fits your Saturday project block + medium pace; each phase $\approx$ 1–2 weeks.

Phase 0 — Setup & data (wk 1–2)

- ☐ Load IBM AMLSim (accounts/transactions/alerts) into SQLite/Postgres
- ☐ Build the account transaction graph (NetworkX)
- ☐ Collect the real AML doc corpus (FATF, FinCEN, RBI)

Phase 1 — Retrieval + data tools = MVP core (wk 3–5)

- ☐ RAG pipeline over AML docs (chunk $\rightarrow$ embed $\rightarrow$ vector store $\rightarrow$ cited retrieval)
- ☐ `get_transaction_history` + `get_transaction_graph` tools
- ☐ Minimal FastAPI backend + one endpoint

Phase 2 — Agent + SAR generation = shippable MVP (wk 6–8)

- ☐ Function-calling agent that plans an investigation and calls tools
- ☐ SAR draft output with citations + rationale
- ☐ Streamlit/Gradio console; **deploy the MVP**

Phase 3 — Wire in FundFlow as a tool (wk 9–10)

- ☐ Retrain/adapt FundFlow's risk engine on AMLSim
- ☐ Expose `score_risk` as an agent tool; fold the score into the SAR

Phase 4 — Eval harness + guardrails (wk 11–12)

- ☐ RAGAS eval + retrieval hit-rate on a held-out set (**report the numbers**)

☐ Guardrails: citation enforcement, insufficient-evidence refusal, hallucination check

☐ `detect_typology` tool

Phase 5 — Deploy hardening + MLOps (wk 13–14)

☐ Dockerize; GitHub Actions CI; logging + feedback capture

☐ Latency / cost pass

Phase 6 — Package (wk 15)

☐ README + architecture diagram + demo video + resume bullets

Differentiator checklist (skip these = just another chatbot)

☐ **Deployed** with a live URL

☐ **Dockerized**

☐ **Agentic** multi-step tool use (not single-shot Q&A)

☐ **RAG eval harness** with reported metrics (RAGAS)

☐ **Guardrails + citations** (no uncited claims)

☐ **MLOps**: CI + logging

☐ **FastAPI backend you can defend cold**

☐ **Real regulatory RAG corpus** (FATF/FinCEN/RBI)

☐ **Honest data disclosure** (synthetic transactions, real regs)

Tech stack

Python · FastAPI · LangGraph/LangChain (function-calling) · an LLM API (GPT-4o-mini / open model) · Chroma or FAISS · sentence-transformers · NetworkX ·

How it'll read on your resume (draft, honest)

Resume A (Finance/AML): Built an agentic AML investigation assistant that auto-drafts Suspicious Activity Reports from transaction-graph analysis + a retrieval layer over real FATF/FinCEN/RBI regulations, integrating an XGBoost risk-scoring engine as a callable tool; deployed with FastAPI + Docker. Eval: RAGAS faithfulness X , retrieval hit-rate Y .

Resume C (MLE/GenAI): Designed a multi-tool LLM agent (function-calling) over a RAG pipeline with a formal evaluation harness (RAGAS) and guardrails; containerized FastAPI service with CI + logging, deployed live.

Interview defense (must explain cold)

- Why **agentic** (multi-step tool use) beats a single RAG call here
 - Your **chunking / embedding / retrieval** choices + how you measured them
 - **RAGAS metrics** — what they mean + your actual numbers
 - How **FundFlow's risk engine** works (you built it — own it end to end)
 - **Guardrails**: how you stop hallucinated / uncited claims
 - **Deployment**: the request path front-end → FastAPI → agent → tools → response
 - **Limitations**: synthetic transactions, and what you'd change with real bank data
-

 **Scope discipline:** a *deployed, working* investigation agent over one dataset + real regs (Phases 0–2) beats a half-built ambitious one. FundFlow-as-a-tool + the eval harness are your differentiators — add them once the MVP ships. Don't gold-plate, and **defend every line**.